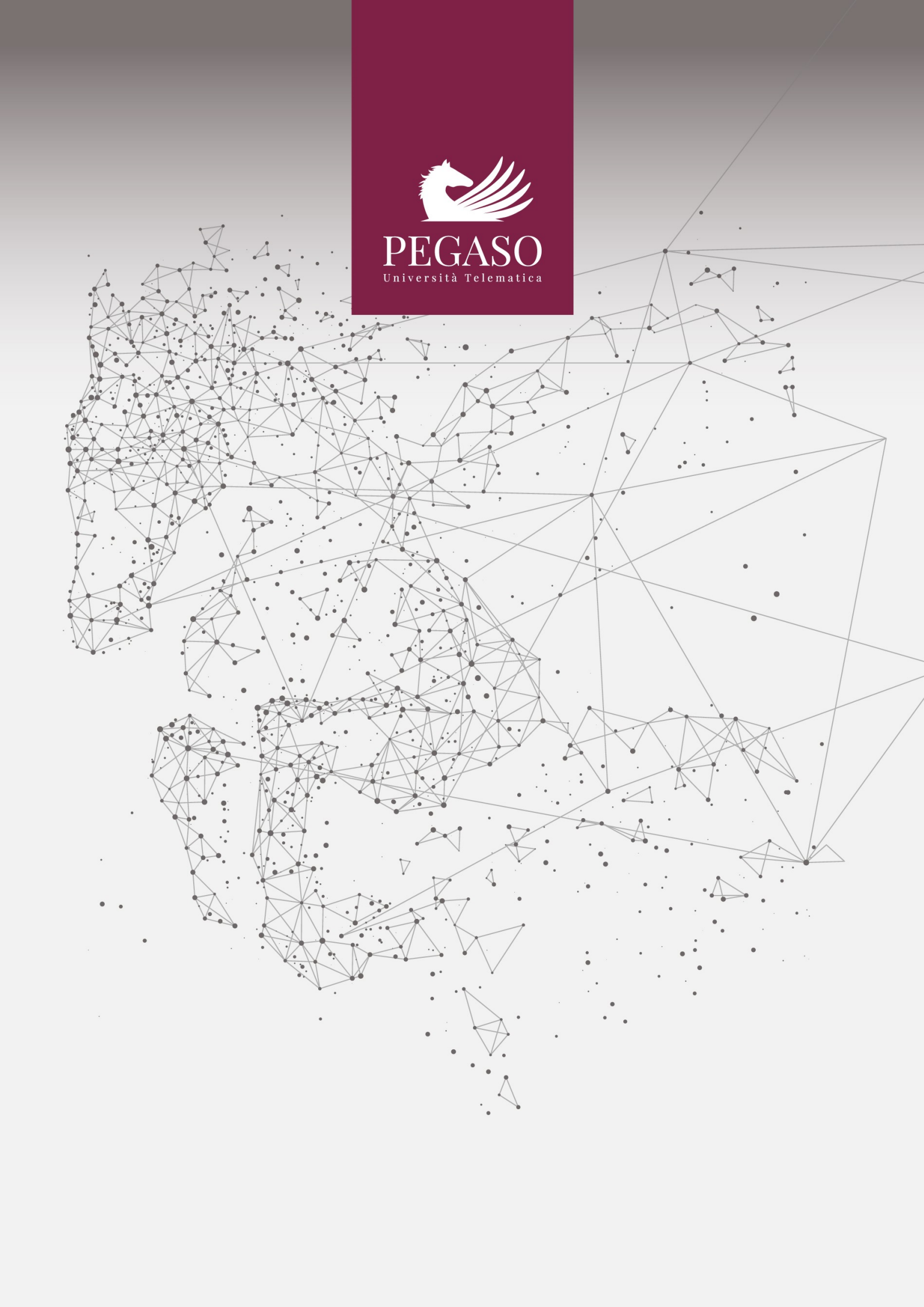

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. NODEJS 3
2. INSTALLAZIONE 5
3. SYNC/ASYNC 9
BIBLIOGRAFIA 12
SITOGRAFIA 13

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Nodejs

Node.js è una piattaforma di sviluppo basata su JavaScript che permette di creare applicazioni web altamente scalabili e performanti. Utilizzando Node.js, è possibile creare applicazioni web in modo asincrono e non bloccante: ciò significa che Node.js può gestire molte richieste contemporaneamente e fornire un'esperienza utente più reattiva.

Per quanto riguarda l'interazione client-server, Node.js permette di creare server web che possono gestire richieste HTTP e WebSocket; inoltre, grazie a Node.js, è possibile eseguire del codice JavaScript lato server, il che significa che è possibile utilizzare lo stesso linguaggio sia lato client che lato server.

Node.js fornisce anche una vasta gamma di moduli NPM, che permettono di estendere le funzionalità della piattaforma e semplificare lo sviluppo. Ad esempio, il modulo "Express" permette di creare rapidamente un'applicazione web utilizzando un sistema di routing, mentre il modulo "Socket.io" permette di utilizzare WebSocket per la comunicazione in tempo reale tra client e server.

I moduli NPM (Node Package Manager) sono pacchetti di codice JavaScript che possono essere facilmente installati e utilizzati in un progetto Node.js: essi consentono di aggiungere funzionalità predefinite ad un'applicazione Node.js senza dover scrivere codice da zero. Ciò può accelerare il processo di sviluppo e rendere più semplice la gestione delle dipendenze di un progetto.

In pratica, i moduli NPM sono pacchetti di codice che possono essere pubblicati su un registro pubblico e scaricati da altri sviluppatori per essere utilizzati nei propri progetti Node.js. Ci sono migliaia di moduli NPM disponibili per diverse funzionalità, come la gestione delle richieste HTTP, la connessione ai database, la gestione delle sessioni utente, la generazione di PDF e molto altro ancora.

Node.js è più efficace rispetto ad altre soluzioni per diversi motivi:

- **Architettura asincrona**: Node.js è basato su un modello di eventi non bloccanti e un'architettura asincrona; questo significa che, invece di creare un thread per ogni richiesta, Node.js utilizza un unico thread per gestire tutte le richieste in arrivo e ciò consente di gestire molte richieste contemporaneamente senza bloccare il thread principale e quindi fornire un'esperienza utente più reattiva. In una soluzione sincrona, ogni richiesta viene gestita in modo sequenziale, una alla volta: ciò significa che il thread principale viene bloccato finché la richiesta corrente non è stata completamente gestita; questo può portare a problemi di prestazioni in caso di un grande volume di richieste in arrivo contemporaneamente (inoltre, le

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

operazioni di I/O lunghe possono causare blocchi e ritardi nell'elaborazione delle richieste successive).

- **Prestazioni elevate**: Node.js è noto per le sue prestazioni elevate grazie alla sua architettura asincrona, inoltre utilizza il motore JavaScript V8 di Google Chrome, che consente di eseguire il codice JavaScript in modo efficiente e veloce.
- **Modularità**: Node.js utilizza un sistema di moduli, il che significa che è possibile utilizzare librerie e moduli di terze parti per estendere le funzionalità del server; inoltre, è possibile creare moduli personalizzati per gestire funzionalità specifiche.
- **Facilità di sviluppo**: Node.js è basato su JavaScript, un linguaggio di scripting ampiamente utilizzato per lo sviluppo web; ciò significa che gli sviluppatori web che conoscono già JavaScript possono facilmente utilizzare Node.js per creare applicazioni web.
- **Scalabilità**: Node.js è altamente scalabile grazie alla sua architettura asincrona; ciò significa che è possibile gestire grandi volumi di traffico web senza che il server si blocchi o si verifichino problemi di prestazioni.

Tradizionalmente, le applicazioni web in tempo reale richiedevano l'uso di tecnologie come WebSockets o AJAX long polling per ottenere un'esperienza utente reattiva. Tuttavia, queste tecnologie sono complesse da implementare e spesso richiedono un notevole sforzo di sviluppo. Node.js è stato sviluppato per semplificare questo processo, utilizzando un modello di eventi non bloccanti e un'architettura asincrona, anche se WebSocket e Socket.IO rimangono ancora le tecnologie principali per gestire la comunicazione bidirezionale in tempo reale tra client e server.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Installazione

Per installare Node.js, è possibile scaricare l'installer dal sito ufficiale di Node.js (https://nodejs.org). L'installer contiene sia il runtime Node.js che il gestore di pacchetti NPM (Node Package Manager).

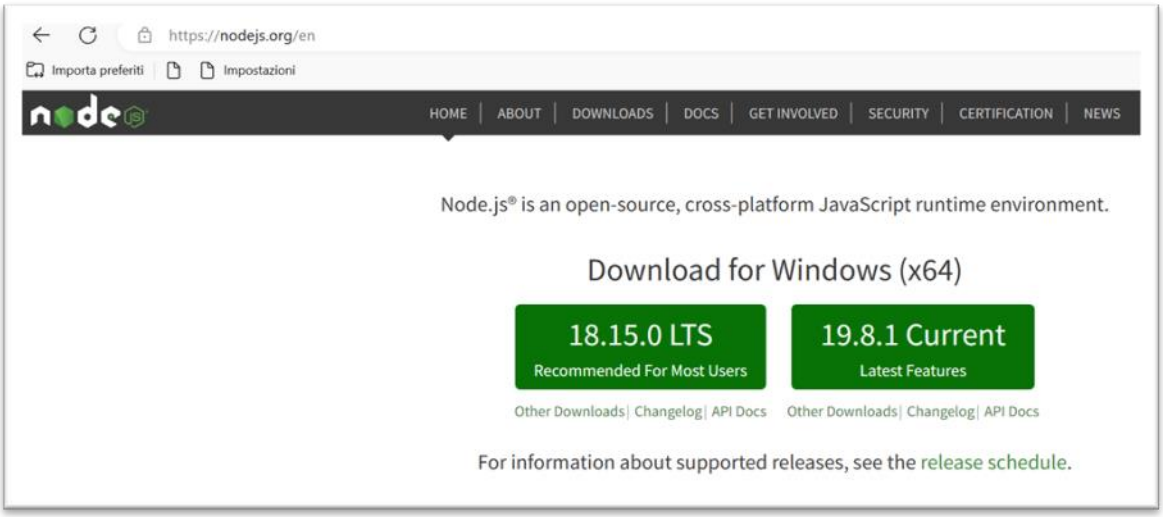


Figura 1 *- Nodejs web site*

Dopo aver installato Node.js, è possibile verificare la versione installata utilizzando il comando node -v. Questo comando visualizza la versione di Node.js installata sulla macchina.

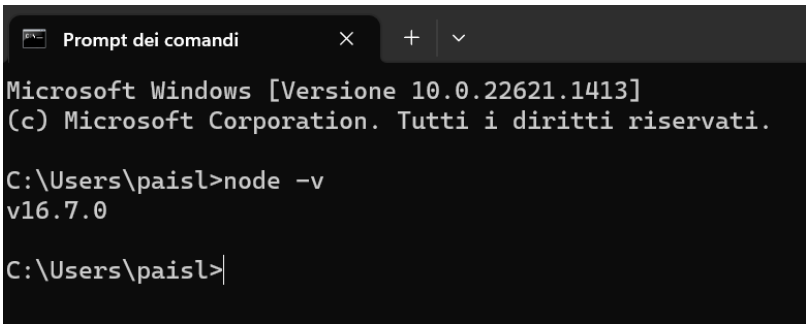


Figura 2 *- node version*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

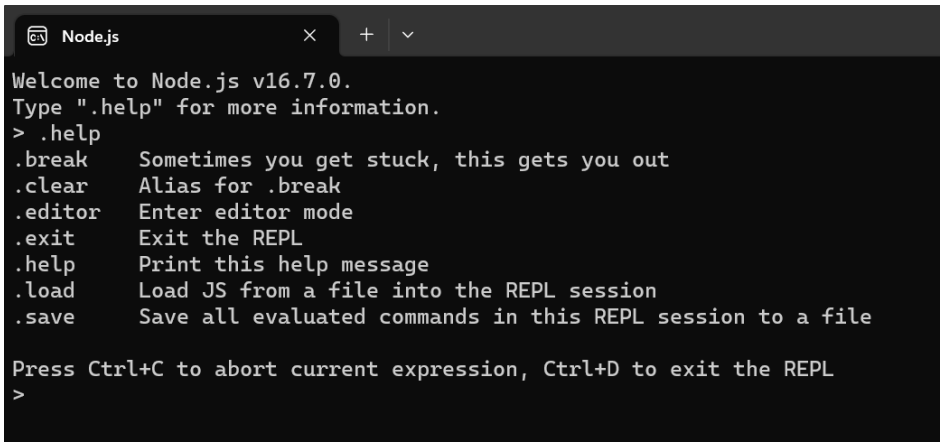


Figura 3 - nodejs prompt

I pacchetti **NPM** sono moduli di codice JavaScript che possono essere installati e utilizzati all'interno di applicazioni Node.js. NPM permette di scaricare e installare i pacchetti NPM, gestire le dipendenze delle applicazioni, e distribuire pacchetti per l'utilizzo in altre applicazioni Node.js.

Per creare una nuova applicazione Node.js, è possibile utilizzare il comando npm init per creare un nuovo file package.json, che contiene le informazioni sulla versione dell'applicazione, le dipendenze e altri metadati. Il comando npm init guida l'utente attraverso la creazione di questo file, richiedendo alcune informazioni come il nome dell'applicazione, la versione, l'autore, etc.

Il file package.json è solitamente posizionato nella directory radice del progetto Node.js; ecco un esempio di un file package.json di base:

```
{
  "name": "my-app",
  "version": "1.0.0",
  "description": "Descrizione del mio progetto",
  "main": "index.js",
  "scripts": {
    "start": "node index.js"
  },
  "dependencies": {
    "express": "^4.17.1"
  }
}
```

In questo esempio, il file package.json definisce le seguenti proprietà:

- **name**: il nome del progetto.
- **version**: la versione del progetto.
- **description**: una descrizione del progetto.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

- **main**: il file principale dell'applicazione, che viene eseguito utilizzando il comando node.
- **scripts**: un oggetto che definisce gli script utilizzabili per l'applicazione. In questo esempio, c'è uno script "start" che esegue il file index.js utilizzando il comando node.
- **dependencies**: un elenco delle dipendenze del progetto. In questo esempio, c'è una dipendenza dal pacchetto express nella versione ^4.17.1, che verrà installato automaticamente quando si esegue il comando npm install.

Una volta creato il file package.json, è possibile installare i pacchetti NPM necessari per l'applicazione utilizzando il comando npm install <nome_pacchetto>: i pacchetti NPM installati vengono salvati nella cartella node_modules all'interno dell'applicazione, e sono elencati nel file package.json come dipendenze.

I pacchetti NPM possono essere utilizzati all'interno di applicazioni Node.js utilizzando il metodo require(). Ad esempio, per utilizzare il pacchetto Express all'interno di un'applicazione Node.js, è possibile utilizzare il seguente codice:

```
const express = require('express');
const app = express();
```

In questo esempio, si utilizza il metodo require() per importare il pacchetto Express, e si crea un'istanza di un oggetto Express utilizzando il metodo express(). L'oggetto app rappresenta l'applicazione Express, e viene utilizzato per definire le rotte e gestire le richieste HTTP.

In generale, per utilizzare un pacchetto NPM in un'applicazione Node.js, è sufficiente installarlo tramite il comando npm install e poi importarlo nel codice utilizzando require(). I pacchetti NPM sono specificati nel file package.json dell'applicazione, che contiene le informazioni sulla versione del pacchetto, le dipendenze e altri metadati dell'applicazione.

In generale, è importante conoscere il concetto di **PATH** quando si lavora con Node.js, in quanto può influire sul corretto funzionamento dell'applicazione e dei pacchetti NPM. Se si riscontrano problemi con l'esecuzione di script Node.js o l'utilizzo di pacchetti NPM, può essere utile verificare il contenuto della variabile PATH e assicurarsi che gli eseguibili necessari siano inclusi nella lista dei percorsi di ricerca.

Il PATH è una variabile d'ambiente che definisce la lista dei percorsi di ricerca per gli eseguibili e i file di sistema. Node.js utilizza il PATH per cercare gli eseguibili necessari all'interno del sistema operativo.

Se Node.js non trovasse un eseguibile, potrebbe essere necessario aggiungere il percorso dell'eseguibile al PATH. Inoltre, quando si utilizza Node.js per eseguire script da linea di comando, potrebbe essere necessario utilizzare il comando npm run per eseguire gli script, invece di utilizzare direttamente il

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

nome dello script: ad esempio, se si ha uno script start definito nel file package.json, il comando per eseguire lo script utilizzando NPM è npm run start; questo comando utilizza il PATH di NPM per cercare l'eseguibile node e quindi eseguire lo script.

Per visualizzare il contenuto della variabile PATH in un sistema Unix-like (come Linux o macOS), è possibile utilizzare il comando echo $PATH. In Windows, è possibile visualizzare il contenuto della variabile PATH utilizzando il comando echo %PATH%.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Sync/Async

In una programmazione sincrona, ogni operazione viene eseguita una dopo l'altra in un unico flusso di esecuzione: questo significa che, quando una funzione viene chiamata, il programma si ferma finché la funzione non ha completato l'operazione e restituito un risultato. Solo a questo punto il programma può passare alla funzione successiva.

In una programmazione asincrona, invece, le operazioni vengono eseguite in modo indipendente dal flusso principale di esecuzione del programma: quando una funzione viene chiamata, il programma non si ferma ma continua a eseguire altre operazioni, mentre la funzione viene eseguita in parallelo. Quando la funzione ha terminato l'operazione, il risultato viene restituito attraverso una callback o una Promise.

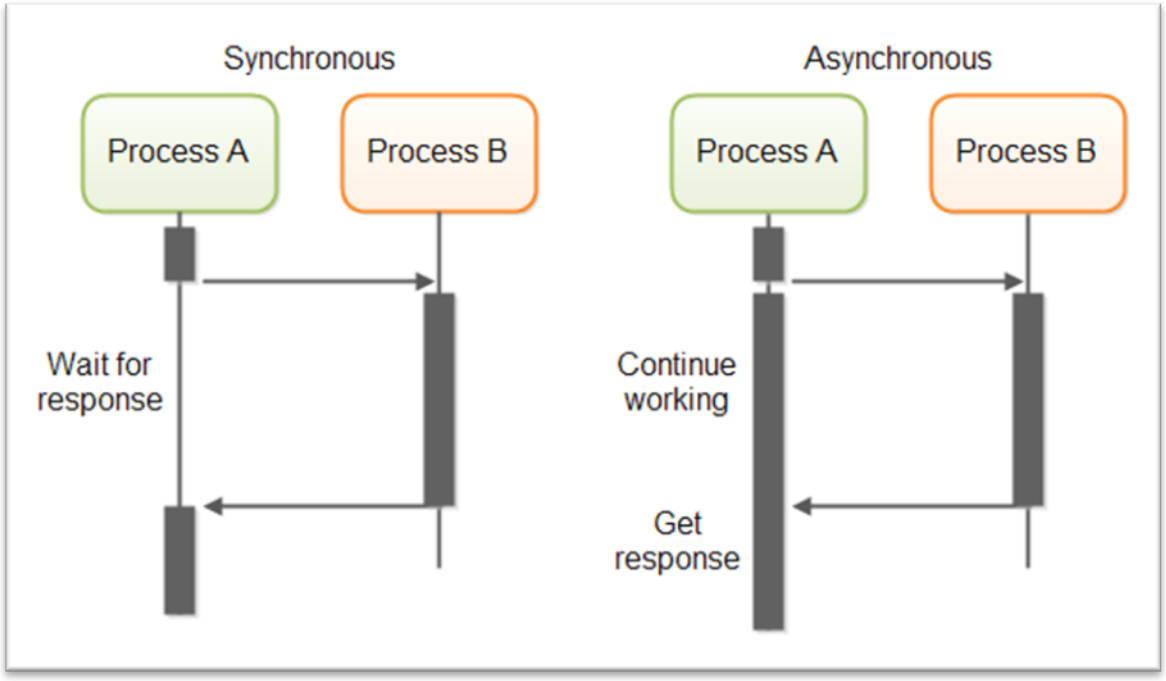


***Figura 4** - Sync / Async*[1]

In JavaScript, la programmazione asincrona è fondamentale per gestire le operazioni che richiedono del tempo, come le richieste HTTP o le operazioni di lettura/scrittura su file system: grazie alla programmazione asincrona, il programma non si blocca durante queste operazioni ma può continuare a eseguire altre attività, migliorando la reattività dell'applicazione e l'esperienza utente.

[1] [1]https://miro.medium.com/v2/resize:fit:828/format:webp/1*Y41dOkntUbR3I4UCJBx9Xg.png

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Per introdurre i concetti di programmazione sincrona e asincrona in JavaScript, possiamo fare un esempio di un'operazione che richiede tempo come la lettura di un file. In JavaScript, la lettura di un file può essere eseguita in modo sincrono o asincrono.

Ecco un esempio di lettura sincrona di un file:

```
const fs = require('fs');
const data = fs.readFileSync('/path/to/file', 'utf-8');
console.log(data);
```

In questo esempio, la lettura del file viene eseguita in modo sincrono, il che significa che il thread di esecuzione del programma viene bloccato finché la lettura non è completata; solo dopo che la lettura è completata, il programma può procedere all'esecuzione del codice successivo.

Ecco invece un esempio di lettura asincrona di un file:

```
const fs = require('fs');
fs.readFile('/path/to/file', 'utf-8', (err, data) => {
  if (err) throw err;
  console.log(data);
});
```

In questo esempio, la lettura del file viene eseguita in modo asincrono, il che significa che il thread di esecuzione del programma non viene bloccato durante la lettura: quando la lettura è completata, viene eseguita la callback passata come parametro alla funzione readFile.

In Node.js, le operazioni di I/O (lettura/scrittura di file, richieste HTTP, ecc.) sono sempre asincrone per non bloccare il thread di esecuzione del programma: questo è uno dei motivi per cui Node.js è così efficiente nel gestire un gran numero di connessioni contemporanee.

async/await è un costrutto introdotto in ES2017 (o ES8) che facilita la programmazione asincrona in JavaScript.

Introduciamo ora i costrutti async / await:

- **async** è utilizzato per dichiarare una funzione asincrona, ovvero una funzione che restituisce una Promise; all'interno di una funzione dichiarata come async, si può utilizzare await per attendere il completamento di un'altra funzione asincrona o di una Promise: in questo modo si può scrivere codice asincrono in modo simile a quello sincrono
- l'utilizzo di **await** sospende l'esecuzione della funzione asincrona in attesa del completamento della Promise: se la Promise viene risolta, await restituisce il valore restituito dalla Promise stessa; se invece la Promise viene rifiutata, viene sollevata un'eccezione.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

Ecco un esempio di utilizzo di async/await:

```
async function getHello() {
  const response = await fetch('https://api.example.com/hello');
  const data = await response.json();
  return data.message;
}

getHello().then(message => {
  console.log(message);
});
```

In questo esempio, la funzione getHello() è marcata come async, il che significa che può contenere la keyword await. All'interno della funzione, viene utilizzato await per attendere la risoluzione della Promise restituita dalla funzione fetch(): questo garantisce che il risultato sia disponibile prima di continuare l'esecuzione della funzione.

Una volta che la Promise è stata risolta, viene eseguito il parsing del risultato come JSON con response.json(), il cui risultato viene a sua volta assegnato alla variabile data.

Infine, viene restituito il valore del messaggio (che presumibilmente si trova all'interno dell'oggetto data), il quale viene loggato nella console una volta che la Promise ritornata dalla funzione getHello() viene risolta; in questo modo, la funzione getHello() può essere utilizzata in modo asincrono con await, garantendo che il risultato sia disponibile quando necessario.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

PEGASO
Università Telematica

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).